

Chapter 8

Policy Gradient Methods

Reading

1. Sutton & Barto, Chapter 9–10, 13
2. Simple random search provides a competitive approach to reinforcement learning at <https://arxiv.org/abs/1803.07055>
3. Proximal Policy Optimization Algorithms <https://arxiv.org/abs/1707.06347>
4. Are Deep Policy Gradient Algorithms Truly Policy Gradient Algorithms? <https://arxiv.org/abs/1811.02553>
5. Asynchronous Methods for Deep Reinforcement Learning <http://proceedings.mlr.press/v48/mnih16.pdf>

This chapter discusses methods to learn the controller that minimizes a given cost functional over trajectories of an unknown dynamical system. We will use what is called the “policy gradient” which will be the main section of this chapter.

Recall from the last chapter that we were able to fit stochastic controllers of the form $u_{\hat{\theta}}(\cdot | x)$ that is a probability distribution on the control-space U for each $x \in X$. We fitted u_{θ} using data from the expert in imitation learning. We did not learn the cost-to-go for the fitted controller, like we did in the lectures on dynamic programming. This is a clever choice: it is often easier to learn the controller in a typical problem than to *compute* the optimal cost-to-go as a parametric function $J^*(x)$.

❓ Can you give another instance when we have computed a controller previously in the class without coming up with its cost-to-go?

8.1 Standard problem setup in RL

Dynamics and rewards In this and the next few chapters we will always consider discrete-time stochastic dynamical systems with a stochastic controller with parameters (weights) θ . We denote them as follows

$$x_{k+1} \sim p(\cdot \mid x_k, u_k) \text{ with noise denoted by } \epsilon_k \\ u_k \sim u_\theta(x_k).$$

We will also change perspective and instead of minimizing the infinite-horizon sum of a runtime cost, maximize the sum of a runtime reward

$$r(x, u) := -q(x, u).$$

We do so simply to conform to tradition and standard notation in reinforcement learning; the two are mathematically completely equivalent. We are interested in maximizing the expected value of the cumulative rewards over infinite-horizon trajectories of the system

$$J(\theta; x_0) = \mathbb{E}_{x_1, x_2, \dots} \left[\underbrace{\sum_{k=0}^{\infty} \gamma^k r(x_k, u_k)}_{\text{discounted return}} \mid x_0 \right]; \quad (8.1)$$

where each $u_k \sim u_\theta(\cdot \mid x_k)$ and each $x_{k+1} \sim p(\cdot \mid x_k, u_k)$.

Trajectory space Let us write out one trajectory of such a system a bit more explicitly. We know that the probability of the next state x_{k+1} given x_k is $p(x_{k+1} \mid x_k, u_k)$. The probability of taking a control u_k at state x_k is $u_\theta(u_k \mid x_k)$. We denote an infinite trajectory by

$$\tau = x_0, u_0, x_1, u_1, \dots$$

The probability of this entire trajectory occurring is

$$p_\theta(\tau) = \prod_{k=0}^{\infty} p(x_{k+1} \mid x_k, u_k) u_\theta(u_k \mid x_k);$$

we have emphasized that the distribution of trajectories depends on the weights of controller θ . If we take the logarithm,

$$\log p_\theta(\tau) = \sum_{k=0}^{\infty} \log p(x_{k+1} \mid x_k, u_k) + \log u_\theta(u_k \mid x_k).$$

Given a trajectory $\tau = x_0, u_0, x_1, u_1, \dots$, the sum

$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r(x_k, u_k) \quad (8.2)$$

is called the discounted return of the trajectory τ . Sometimes we will also talk of the undiscounted return of the trajectory which is the sum of the rewards up to some fixed finite horizon T without the discount factor pre-multiplier. Using this notation, we can write out objective from (8.1) as

$$J(\theta; x_0) = \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau) \mid x_0] \quad (8.3)$$

where $p(\tau)$ is the probability distribution of an infinitely long trajectory τ .

Observe what is probably the most important point in policy-gradient based reinforcement learning: the probability of trajectory is an infinite product of terms. All terms are smaller than 1 (they are probabilities), so it is essentially zero even if the state-space and the control-space are finite (even if they are small). Any given infinite (or long) trajectory is quite rare under the probability distribution of the stochastic controller. Policy-gradient methods sample lots of trajectories from the system and average the returns across these trajectories. Since the set of trajectories of even a small MDP is so large, sampling lots of trajectories, or even the most likely ones, is also very hard. This is a key challenge in getting RL algorithms to work.

Our goal in this chapter is to compute the best stochastic controller which maximizes the average discounted return. Mathematically, this amounts to finding

$$\hat{\theta} = \operatorname{argmax}_{\theta} J(\theta; x_0) := \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau) \mid x_0]. \quad (8.4)$$

The objective $J(\theta)$ is called the average return of the controller u_θ .

Computing the average return $J(\theta)$ Before we move on to optimizing $J(\theta)$, let us discuss how to compute it for given weights θ of the stochastic controller. We can sample n trajectories from the system and compute the an estimate of the expectation

$$\hat{J}(\theta) \approx \frac{1}{n} \sum_{i=0}^n \sum_{k=0}^T \gamma^k r(x_k^i, u_k^i) \quad (8.5)$$

for some large time-horizon T and where each $u_k^i \sim u_\theta(\cdot \mid x_k^i)$.

8.2 Cross-Entropy Method (CEM)

Let us first consider a simple method to compute the best controller. The basic idea is to solve the problem

$$\hat{\theta} = \operatorname{argmax}_{\theta} J(\theta)$$

🔍 Contrast (8.5) with the complexity of policy evaluation which was simply a system of linear equations. Evaluating the policy without having access to the dynamical system is harder.

58 using gradient descent. We would like to update weights θ iteratively

$$\theta^{(k+1)} = \theta^{(k)} + \eta \nabla J(\theta).$$

59 where the step-size is $\eta > 0$ and $\nabla J(\theta)$ is the gradient of the objective
60 $J(\theta)$ with respect to weights θ . Instead of computing the exact $\nabla J(\theta)$
61 which we will do in the next section, let us simply compute the gradient
62 using a finite-difference approximation. The i^{th} entry of the gradient is

$$(\widehat{\nabla} J(\theta))_i = \frac{J(\theta + \epsilon e_i) - J(\theta - \epsilon e_i)}{2\epsilon} \approx \frac{\widehat{J}(\theta + \epsilon e_i) - \widehat{J}(\theta - \epsilon e_i)}{2\epsilon}.$$

63 where $e_i = [0, 0, \dots, 0, 1, 0, \dots]$ is a vector with 1 on the i^{th} entry. Each
64 quantity $\widehat{J}$ is computed as the empirical average return of n trajectories
65 from the system. We compute all entries of the objective using this
66 approximation and update the parameters using

$$\theta^{(k+1)} = \theta^{(k)} + \eta \widehat{\nabla} J(\theta^{(k)}).$$

67 **A more efficient way to compute the gradient using finite-differences**
68 Instead of picking perturbations e_i along the cardinal directions, let us
69 sample them from a Gaussian distribution

$$\xi^i \sim N(0, \sigma^2 I)$$

70 for some user-chosen covariance σ^2 . We can however no longer use
71 the finite-difference formula to compute the derivative because the noise
72 e is not aligned with the axes. We can however use a Taylor series
73 approximation as follows. Observe that

$$J(\theta + \xi) \approx J(\theta) + \langle \nabla J(\theta), \xi \rangle$$

74 where $\langle \cdot, \cdot \rangle$ is the inner product. Given m samples $\xi^1, \dots, \xi^m$ observe
75 that

$$\begin{aligned} \widehat{J}(\theta + \xi^1) &= \widehat{J}(\theta) + \langle \nabla J(\theta), \xi^1 \rangle \\ \widehat{J}(\theta + \xi^2) &= \widehat{J}(\theta) + \langle \nabla J(\theta), \xi^2 \rangle \\ &\vdots \\ \widehat{J}(\theta + \xi^m) &= \widehat{J}(\theta) + \langle \nabla J(\theta), \xi^m \rangle. \end{aligned} \tag{8.6}$$

76 is a linear system of equations in $\nabla J(\theta) \in \mathbb{R}^p$ where $\theta \in \mathbb{R}^p$. All
77 quantities $\widehat{J}$ are estimated as before using trajectories drawn from the
78 system. We solve this linear system, e.g., using least-squares if $m > p$, to
79 get an estimate of the gradient $\widehat{\nabla} J(\theta) \in \mathbb{R}^p$.

The Cross-Entropy Method is a more crude but simpler way to

implement the above least-squares formulation. At each iteration it updates the parameters using the formula

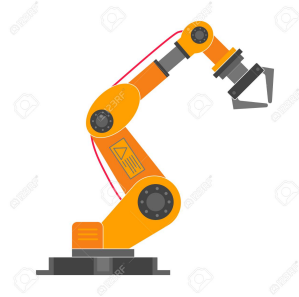
$$\theta^{(k+1)} = \mathbb{E}_{\theta \sim N(\theta^{(k)}, \sigma^2 I)} \left[\theta \mathbf{1}_{\{\hat{J}(\theta) > \hat{J}(\theta^{(k)})\}} \right]. \quad (8.7)$$

In simple words, the CEM samples a few stochastic controllers u_θ from a Gaussian (or any other distribution) centered around the current controller $u_{\theta^{(k)}}$ and updates the weights θ^k in a direction that leads to an increase in $\hat{J}(\theta) > \hat{J}(\theta^{(k)})$.

8.2.1 Some remarks on sample complexity of simulation-based methods

CEM may seem to be a particularly bad method to maximize $J(\theta)$, after all we are perturbing the weights of the stochastic controller randomly and updating the weights if they result in a better average return $\hat{J}(\theta)$. This is likely to work well if the dimensionality of weights $\theta \in \mathbb{R}^p$, i.e., p , is not too large. But is unlikely to work well if we are sampling θ in high-dimensions. Typical applications are actually the latter, remember that we are interested in using a deep network as a stochastic controller and θ are the weights of the neural networks.

Let us do a quick computation, if the state is $x \in \mathbb{R}^d$ and $u \in \mathbb{R}^m$ with $d = 12$ (joint angles and velocities) and $m = 6$ for a six-degree of freedom robot manipulator



and if we use a two-layer neural network with 64 neurons in the hidden layer, the total number of weights $\theta \in \mathbb{R}^p$ for the function $u_\theta(\cdot | x) = N(\mu_\theta(x), \sigma_\theta^2(x)I)$ where $\sigma^2(x)$ is a vector in $\mathbb{R}^m$, is

$$\begin{aligned} p &= \underbrace{(12 \times 64 + 64)}_{\text{for } \mu_\theta(x)} + \underbrace{(12 \times 64 + 64)}_{\text{for } \sigma_\theta^2(x)} + (64 \times 6 + 6) \\ &= 2,444. \end{aligned}$$

This is a very high-dimensional space to sample exhaustively. Note that it is quite large even if the input and output dimensions of the neural network are not too large. To appreciate the complexity of computing the gradient $\nabla J(\theta)$, let us think of how to compute it using finite-differences.

We need two estimates $\hat{J}(\theta - \epsilon e_i)$ and $\hat{J}(\theta + \epsilon e_i)$ for every dimension $i \in \{1, \dots, p\}$. Each estimate requires us to obtain n trajectories from the system. Since the number of trajectories that a robot can take is quite diverse, we should use a large n , so let's pick $n = 100$. The total number of trajectories required to update the parameters $\theta^{(k)}$ at each iteration is

$$2 p n \approx 10^6.$$

This is an absurdly large number, and things are even more daunting when we realize that each update of the weights requires us to sample these many trajectories from the system. It is not reasonable to sample such a large number of trajectories from an actual robot, that too for each update of the weights.

Using fast simulators for RL If we expand our horizon and think of learning controllers in simulation, things feel much more reasonable. While running a large number of trajectories may degrade a real robot beyond use, doing so requires just computation time in a robot simulator. There is a large number of simulators that are available with various capabilities, e.g., Gazebo (<http://gazebo.org>) is a sophisticated simulator inside ROS that uses a number of Physics engines such as Bullet (<https://pybullet.org/wordpress>), MuJoCo (<http://www.mujoco.org>) is incredibly fast although not very good modeling contact, Unity is a popular platform to simulate driving and more complicated scenes (<https://docs.nvidia.com/isaac/isaac/doc/simulation/unity3d.html>), Drake (<https://drake.mit.edu>) is better at contact modeling but more complex and slower. Most robotics companies develop their own driving simulators in-house. The assigned reading (#2) for this chapter is a paper which develops a very fast implementation of the CEM for use in simulators.

Working well in simulation does not mean that a controller works well on the real robot It is important to realize that a simulator is not equivalent to the physical robot. Each simulator makes certain trade-offs in capturing the dynamics of the real system and it is not a given that a controller that was learned using data from a simulator will work well on a real robot. For instance, OpenAI had to develop a large number of tricks (which took about a year) to modify the simulator to enable the learned policy to work well on a robot (<https://openai.com/blog/learning-dexterity>) for a fairly narrow set of tasks.

i For comparison, a busy espresso bar in a city makes about 500 shots per day. The espresso machine would have to work for 5 years *without breaking down* to make 10^6 shots.

8.3 The Policy Gradient

In this section, we will study how to take the gradient of the objective $J(\theta)$, without using finite-differences.

136 We would like to solve the optimization problem

$$\max_{\theta} J(\theta) := \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau) \mid x_0]$$

137 We will suppress the dependence on x_0 to keep the notation clear. The
 138 expectation is taken over all trajectories starting at state x_0 realized using
 139 the stochastic controller $u_{\theta}(\cdot \mid x)$. We to update weights θ using gradient
 140 descent which amounts to

$$\theta^{(k+1)} = \theta^{(k)} + \eta \nabla_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau)].$$

141 First let us note that the distribution p_{θ} using which we compute the
 142 expectation also depends on the weights θ . This is why we cannot simply
 143 move the derivative ∇_{θ} inside the expectation

$$\nabla_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau)] \neq \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} R(\tau)].$$

144 We need to think of a new technique to compute the gradient above.
 145 Essentially, we would like to do the chain rule of calculus but where
 146 one of the functions in the chain is an expectation. The likelihood-ratio
 147 trick described next allows us to take such derivatives. Here is how the
 148 computation goes

$$\begin{aligned} \nabla_{\theta} \mathbb{E}_{\tau \sim p_{\theta}} [R(\tau)] &= \nabla_{\theta} \int R(\tau) p_{\theta}(\tau) d\tau \\ &= \int R(\tau) \nabla_{\theta} p_{\theta}(\tau) d\tau \\ &\quad (\text{move the gradient inside, integral is over trajectories } \tau \text{ which do not depend on } \theta \text{ themselves}) \\ &= \int R(\tau) p_{\theta}(\tau) \frac{\nabla p_{\theta}(\tau)}{p_{\theta}(\tau)} d\tau \\ &= \int R(\tau) p_{\theta}(\tau) \nabla \log p_{\theta}(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau) \nabla \log p_{\theta}(\tau)] \\ &\approx \frac{1}{n} \sum_{i=1}^n R(\tau^i) \nabla \log p_{\theta}(\tau^i) \end{aligned} \tag{8.8}$$

149 This is called the likelihood-ratio trick to compute the policy gradient. It
 150 simply multiplies and divides by the term $p_{\theta}(\tau)$ and rewrites the term
 151 $\frac{\nabla p_{\theta}}{p_{\theta}} = \nabla \log p_{\theta}$. It gives us a neat way to compute the gradient: we
 152 sample n trajectories $\tau^1, \dots, \tau^n$ from the system and average the return
 153 of each trajectory $R(\tau^i)$ weighted by the gradient of the likelihood of
 154 taking each trajectory $\log p_{\theta}(\tau^i)$. The central point to remember here is

155 that the gradient

$$\begin{aligned}
 \nabla_{\theta} \log p_{\theta}(\tau^i) &= \nabla_{\theta} \sum_{k=0}^T \log p(x_{k+1}^i | x_k^i, u_k^i) + \log u_{\theta}(u_k^i | x_k^i) \\
 &= \sum_{k=0}^T \nabla_{\theta} \log u_{\theta}(u_k^i | x_k^i)
 \end{aligned}
 \tag{8.9}$$

156 is computed using backpropagation for a neural network. This expression
 157 is called the policy gradient because it is the gradient of the objective $J(\theta)$
 158 with respect to the parameters of the controller/policy θ .

159 **Variance of policy gradient** The expression for the policy gradient may
 160 seem like a sleight of hand. It is a clean expression to get the gradient of
 161 the objective but also comes with a number of problems. Observe that

$$\begin{aligned}
 \nabla_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau)] &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[R(\tau) \frac{\nabla p_{\theta}(\tau)}{p_{\theta}(\tau)} \right] \\
 &\approx \frac{1}{n} \sum_{i=1}^n R(\tau^i) \frac{\nabla p_{\theta}(\tau^i)}{p_{\theta}(\tau^i)}.
 \end{aligned}$$

162 If we sample trajectories τ^i that are not very likely under the distribution
 163 $p_{\theta}(\tau)$, the denominator in some of the summands can be very small.
 164 For trajectories that are likely, the denominator is large. The empirical
 165 estimate of the expectation using n trajectories where some terms are
 166 very small and some others very large, therefore has a large variance. So
 167 one does need lots of trajectories from the system/simulator to compute a
 168 reasonable approximation of the policy gradient.

169 8.3.1 Reducing the variance of the policy gradient

170 **Control variates** You will perhaps appreciate that computing the accu-
 171 rate policy gradient is very hard in practice. Control variates is a general
 172 concept from the literature on Monte Carlo integration and is typically
 173 introduced as follows. Say we have a random variable X and we would
 174 like to guess its expected value $\mu = \mathbb{E}[X]$. Note that X is an unbiased
 175 estimator of μ but it may have a large variance. If we have another random
 176 variable Y with known expected value $\mathbb{E}[Y]$, then

$$\hat{X} = X + c(Y - \mathbb{E}[Y]) \tag{8.10}$$

177 is also an unbiased estimator for μ for any value of c . The variance of $\hat{X}$ is

$$\text{Var}(\hat{X}) = \text{Var}(X) + c^2 \text{Var}(Y) + 2c \text{Cov}(X, Y).$$

178 which is minimized for

$$c^* = -\frac{\text{Cov}(X, Y)}{\text{Var}(Y)}$$

179 for which we have

$$\text{Var}(\hat{X}) = \text{Var}(X) - c^{*2} \text{Var}(Y) = (1 - \text{Corr}^2(X, Y)) \text{Var}(X).$$

180 By subtracting $Y - \mathbb{E}[Y]$ from our observed random variable X , we have
 181 reduced the variance of X if the correlation between X and Y is non-zero.
 182 Most importantly, note that no matter what Y we plug into the above
 183 expression, we can never increase the variance of X ; the worst that can
 184 happen is that we pick a Y that is completely uncorrelated with X and
 185 end up achieving nothing.

186 **Baseline** We will now use the concept of a control variate to reduce the
 187 variance of the policy gradient. This is known as “building a baseline”.
 188 The simplest baseline one can build is to subtract a constant value from
 189 the return. Consider the PG given by

$$\begin{aligned} \nabla J(\theta) &= \mathbb{E}_{\tau \sim p_\theta} [R(\tau) \nabla \log p_\theta(\tau)] \\ &= \mathbb{E}_{\tau \sim p_\theta(\tau)} [(R(\tau) - b) \nabla \log p_\theta(\tau)]. \end{aligned}$$

190 Observe that

$$\begin{aligned} \mathbb{E}_{\tau \sim p_\theta(\tau)} [b \nabla \log p_\theta(\tau)] &= \int d\tau b p_\theta(\tau) \nabla \log p_\theta(\tau) \\ &= \int d\tau b \nabla p_\theta(\tau) = b \nabla \int d\tau p_\theta(\tau) = b \nabla 1 = 0. \end{aligned}$$

191 **Example 1: Using the average returns of a mini-batch as the baseline**
 192 What is the simplest baseline b we can cook up? Let us write the mini-batch
 193 version of the policy gradient

$$\hat{\nabla} J(\theta) := \frac{1}{\ell} \sum_{i=1}^{\ell} [R(\tau^i) \nabla \log p_\theta(\tau^i)].$$

194 where $\tau^1, \dots, \tau^\ell$ are trajectories that are a part of our mini-batch. We can
 195 set

$$b = \frac{1}{\ell} \sum_{i=1}^{\ell} R(\tau^i)$$

196 can use the variance-reduced gradient

$$\hat{\nabla} J(\theta) = \frac{1}{\ell} \sum_{i=1}^{\ell} [(R(\tau^i) - b) \nabla \log p_\theta(\tau^i)].$$

197 This is a one-line change in your code for policy gradient so there is no
 198 reason not to do it.

199 **Example 2: A weighted averaged of the returns using the log-likelihood**
 200 **of the trajectory** The previous example showed how we can use one

constant baseline, namely the average of the discounted returns of all trajectories in a mini-batch. What is the **best** constant b we can use? We can perform a similar computation as done in the control variate introduction to minimize the variance of the policy gradient to get the following.

$$\begin{aligned}\delta\left(\widehat{\nabla}_{\theta_i} J(\theta)\right) &= \mathbb{E}_{\tau}\left[\left((R(\tau) - b_i) \nabla_{\theta_i} \log p_{\theta}(\tau)\right)^2\right] - \left(\mathbb{E}_{\tau}\left[\left((R(\tau) - b_i) \nabla_{\theta_i} \log p_{\theta}(\tau)\right)\right]\right)^2 \\ &= \mathbb{E}_{\tau}\left[\left((R(\tau) - b_i) \nabla_{\theta_i} \log p_{\theta}(\tau)\right)^2\right] - \left(\widehat{\nabla}_{\theta_i} J(\theta)\right)^2.\end{aligned}$$

Set

$$\frac{\delta\left(\widehat{\nabla}_{\theta_i} J(\theta)\right)}{db_i} = 0$$

in the above expression to get

$$b_i = \frac{\mathbb{E}_{\tau}\left[(\nabla_{\theta_i} \log p_{\theta}(\tau))^2 R(\tau)\right]}{\mathbb{E}_{\tau}\left[(\nabla_{\theta_i} \log p_{\theta}(\tau))^2\right]}$$

which is the baseline you should subtract from the gradient of the i^{th} parameter θ_i to result in the largest variance reduction. This expression is just the expected return but it is weighted by the magnitude of the gradient, this again is 1–2 lines of code.

🔗 Show that any function that only depends on the state x that can be written as a potential (i.e., whose derivative, the force, is conservative) can be used as a baseline in the policy gradient. This technique is known as reward shaping.

8.4 An alternative expression for the policy gradient

We will first define an important quantity that helps us think of RL algorithms.

Definition 8.1 (Discounted state visitation frequency). Given a stochastic controller $u_{\theta}(\cdot \mid x)$ the discounted state visitation frequency for a discrete-time dynamical system is given by

$$d^{\theta}(x) = \sum_{k=0}^{\infty} \gamma^k \mathbb{P}(x_k = x \mid x_0, u_k \sim u_{\theta}(\cdot \mid x_k)).$$

The distribution $d^{\theta}(x)$ is the probability of visiting a state x computed over all trajectories of the system that start at the initial state x_0 . If $\gamma = 1$, this is the steady-state distribution of the Markov chain underlying the Markov Decision Process where at each step the MDP chooses the control $u_k \sim u_{\theta}(\cdot \mid x_k)$. The fact that we have defined the discounted distribution is a technicality; this version is seen in the policy gradient expression. You will also notice that $d^{\theta}(x)$ is not a normalized distribution. The normalization constant is difficult to characterize both theoretically and empirically and we will not worry about it here; RL algorithms do not require it.

Q-function Using the discounted state visitation frequency, the policy gradient that we saw before can be written in terms of the value function as follows.

$$\begin{aligned}\nabla J(\theta) &= \mathbb{E}_{\tau \sim p_\theta} [R(\tau) \nabla \log p_\theta(\tau)] \\ &= \mathbb{E}_{x \sim d^\theta} \mathbb{E}_{u \sim u_\theta(\cdot | x)} [q^\theta(x, u) \nabla_\theta \log u_\theta(u | x)].\end{aligned}\quad (8.11)$$

The function $q^\theta(x, u)$ is similar to the cost-to-go that we have studied in dynamic programming and is called the Q-function

$$q^\theta(x, u) = \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau) | x_0 = x, u_0 = u]. \quad (8.12)$$

It is the infinite-horizon discounted cumulative reward (i.e., the return) if the system starts at state x and takes the control u in the first step and runs the controller $u_\theta(\cdot | x)$ for all steps thereafter. We make the dependence of q^θ on the parameters θ of the controller explicit.

i The derivation of this expression is easy although tedious, you can find it in the Appendix of the paper “Policy gradient methods for reinforcement learning with function approximation” at <https://papers.nips.cc/paper/1713-policy-gradient-methods-for-reinforcement-learning-with-function-approximation.pdf>.

Compare the above formula for the policy gradient with the one we had before in (8.8)

$$\begin{aligned}\hat{\nabla} J(\theta) &= \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau) \nabla \log p_\theta(\tau)] \\ &= \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{k=0}^T \gamma^k r(x_k, u_k) \right) \left(\sum_{k=0}^T \nabla \log u_\theta(u_k | x_k) \right) \right].\end{aligned}$$

It is important to notice that this is an expectation over trajectories; whereas (8.11) is an expectation over states x sampled from the discounted state visitation frequency. The control u_k for both is sampled from the stochastic controller at each time-step k . The most important distinction is that (8.11) involves the expectation of the Q-function q^θ weighted by the gradient of the log-likelihood of picking each control action. There are numerous hacky ways of deriving (8.11) from (8.8) but remember that they are fundamentally different expressions of the *same policy gradient*.

This expression allows understanding of a number of properties of reinforcement learning.

1. While the algorithm collects the data, states that are unlikely under the distribution d^θ contribute little to (8.11). In other words, the policy gradient is insensitive to such states. The policy update will not consider these unlikely states that the system is prone to visit infrequently using the controller u_θ .
2. The opposite happens for states which are very likely. For two controls u_1, u_2 at the same state x , the policy increases the log-likelihood of taking the controls weighted by their values $q^\theta(x, u_1)$ and $q^\theta(x, u_2)$. This is sort of the “definition” of reinforcement learning. In the expression (8.8) the gradient was increasing the

likelihood of trajectories with high returns, here it deals with states and controls individually.

8.4.1 Implementing the new expression

Suppose we have a stochastic control that is a Gaussian

$$u_\theta(u \mid x) = \frac{1}{(2\pi\sigma^2)^{p/2}} e^{-\frac{\|u - \theta^\top x\|^2}{2\sigma^2}}$$

where $\theta \in \mathbb{R}^{d \times p}$ and $u \in \mathbb{R}^p$; the variance σ can be chosen by the user. We can easily compute $\log u_\theta(u \mid x)$ in (8.11). How should one compute $q^\theta(x, u)$ in (8.12)? We can again estimate it using sample trajectories from the system; each of these trajectories would have to start from a state x and the control at the first step would be u , with the controller u_θ being used thereafter. Note that we have one such trajectory, namely the remainder of the trajectory where we encountered (x, u) while sampling trajectories for the policy gradient in (8.11). In practice, we do not sample trajectories a second time, we simply take this trajectory, let us call it $\tau_{x,u}$ and set

$$q^\theta(x, u) = \sum_{k=0}^T \gamma^k r(x_k, u_k)$$

for some large time-horizon T where $(x_0, u_0) = (x, u)$ and the summation is evaluated for (x_k, u_k) that lie on the trajectory $\tau_{x,u}$. Effectively, we are evaluating (8.12) using one sample trajectory, a highly erroneous estimate of q^θ .

8.5 Actor-Critic methods

We can of course do more sophisticated things to evaluate the Q-function q^θ in our new expression of the policy gradient.

Actor-Critic methods fit a Q-function to the data collected from the system using the current controller (policy evaluation step) and then use this fitted Q-function in the expression of the policy gradient (8.11) to update the controller. In this sense, Actor-Critic methods are conceptually similar to policy iteration.

In order to understand how to fit the Q-function, first recall that it should satisfy the Bellman equation. This means

$$q^\theta(x, u) = r(x, u) + \gamma \mathbb{E}_{u \sim u_\theta(\cdot \mid x'), x' \sim P(\cdot \mid x, u)} [q^\theta(x', u')]. \quad (8.13)$$

We do not know a model for the system so we cannot evaluate the expectation over $x' \sim P(\cdot \mid x, u)$ like we used to in dynamic programming. But we do have the ability to get trajectories τ^i from the system.

Let's say (x_k^i, u_k^i) lie on τ^i at time-step k . We can then estimate the expectation over $P(\cdot | x_k^i, u_k^i)$ using simply x_{k+1}^i and the expectation over the controls using simply u_{k+1}^i to write

$$q^\theta(x_k^i, u_k^i) \approx r(x_k^i, u_k^i) + \gamma q^\theta(x_{k+1}^i, u_{k+1}^i) \quad \text{for all } i \leq n, k \leq T.$$

This is a nice constraint on the Q-function. If this were a discrete-state, discrete-control MDP, it is a set of linear equations for the q-values. These constraints would be akin to our linear equations for evaluating a policy in dynamic programming except that instead of using the dynamics model (the transition matrix), we are using trajectories sampled from the system.

Parameterizing the Q-function using a neural network If we are dealing with a continuous state/control-space, we can think of parameterizing the q-function using parameters φ

$$q_\varphi^\theta(x, u) : X \times U \rightarrow \mathbb{R}.$$

The parameterization is similar to the parameterization of the controller, e.g., just like we would write a deterministic controller as

$$u_\theta(x) = \theta^\top x$$

we can think of a linear Q-function of the form

$$q_\varphi^\theta(x, u) = \varphi^\top \begin{bmatrix} x \\ u \end{bmatrix}, \quad \varphi \in \mathbb{R}^{m+d}$$

which is a linear function in the states and controls. You can also think of using something like

$$q_\varphi^\theta(x, u) = \begin{bmatrix} 1 & x & u \end{bmatrix} \varphi \begin{bmatrix} 1 \\ x \\ u \end{bmatrix} \quad \varphi \in \mathbb{R}^{(m+d+1) \times (m+d+1)}.$$

which is quadratic in the states and controls, or in general a deep network with weights φ as the Q-function.

Fitting the Q-function We can now “fit” the parameters of the Q-function by solving the problem

$$\hat{\varphi} = \underset{\varphi}{\operatorname{argmin}} \frac{1}{n(T+1)} \sum_{i=1}^n \sum_{k=0}^T \|q_\varphi^\theta(x_k^i, u_k^i) - r(x_k^i, u_k^i) - \gamma q_\varphi^\theta(x_{k+1}^i, u_{k+1}^i)\|^2. \quad (8.14)$$

which is nothing other than enforcing the Bellman equation in (8.13). If the Q-function is linear in $[x, u]$ this is a least squares problem, if it is quadratic the problem is a quadratic optimization problem which can also be solved efficiently, in general we would solve this problem using stochastic gradient descent if we are parameterizing the Q-function using a deep network. Such a Q-function is called the “critic” because it *evaluates*

the controller u_θ , or the “actor”. This version of the policy gradient where one fits the parameters of both the controller and the Q-function are called Actor-Critic methods.

Actor-Critic Methods We fit a deep network with weights θ to parameterize a stochastic controller $u_\theta(\cdot | x)$ and another deep network with weights φ to parameterize the Q-function of this controller, $q_\varphi^\theta(x, u)$. Let the controller weights at the k^{th} iteration be $\theta^{(k)}$ and the Q-function weights be φ^k .

1. Sample n trajectories, each of T timesteps, using the current controller $u_\theta^{(k)}(\cdot | x)$.
2. Fit a Q-function $q_{\varphi^{k+1}}^{\theta^{(k)}}$ to this data using (8.14). using stochastic gradient descent. While performing this fitting (although it is not mathematically sound), it is common to use initialize the Q-function weights to their values from the previous iteration φ^k .
3. Compute the policy gradient using the alternative expression in (8.11) and update parameters of the policy to $\theta^{(k+1)}$.

❗ We will be pedantic and always write the q -function as q_φ^θ . The superscript θ denotes that this is the q -function corresponding to the θ -parameterized controller u_θ . The subscript denotes that the q -function is parameterized by parameters φ .

8.5.1 Advantage function

The new expression for the policy gradient in (8.11) also has a large variance; this should be no surprise, it is after all equal to the old expression. We can however perform variance reduction on this using the value function.

Our goal as before would be construct a relevant baseline to subtract from the Q-function. It turns out that any function that depends only upon the state x and is a potential, i.e., whose gradient has zero curl, is a valid baseline. This gives a powerful baseline for us to use in policy gradients. We can use the value function as the baseline. The value function for controls taken by the controller $u_\theta(\cdot | x)$ (notice that this is not the optimal value function, it is simply the policy evaluation) is given by

$$v^\theta(x) = \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau) | x_0 = x]$$

where $u_k \sim u_\theta(\cdot | x_k)$ at each timestep. We also know that the value function is the expected value of the Q-function across different controls sampled by the controller

$$v^\theta(x) = \mathbb{E}_{u \sim u_\theta(\cdot | x)} [q^\theta(x, u)]. \quad (8.15)$$

The value function again satisfies the dynamic programming principle/-

322 Bellman equation

$$v^\theta(x) = \mathbb{E}_{u \sim u_\theta(\cdot|x)} \left[r(x, u) + \gamma \mathbb{E}_{x' \sim \mathcal{P}(\cdot|x, u)} [v^\theta(x')] \right].$$

323 We again parameterize the value function

$$v_\psi^\theta(x) : X \rightarrow \mathbb{R}$$

324 using parameters ψ and fit it to the data in the same way as (8.14) to get

$$\hat{\psi} = \underset{\psi}{\operatorname{argmin}} \frac{1}{n(T+1)} \sum_{i=1}^n \sum_{k=0}^T \|v_\psi^\theta(x_k^i) - r(x_k^i, u_k^i) - \gamma v_\psi^\theta(x_{k+1}^i)\|^2. \quad (8.16)$$

Using this baseline can modify the policy gradient to be

$$\nabla J(\theta) = \mathbb{E}_{x \sim d^\theta} \mathbb{E}_{u \sim u_\theta(\cdot|x)} \left[\underbrace{\left(q_\varphi^\theta(x, u) - v_\psi^\theta(x) \right)}_{a_{\varphi, \psi}^\theta(x, u)} \nabla_\theta \log u_\theta(u | x) \right]. \quad (8.17)$$

where each of the functions q_φ^θ and v_ψ^θ are themselves fitted using (8.14) and (8.16) respectively. The difference

$$\begin{aligned} a_{\varphi, \psi}^\theta(x, u) &= q_\varphi^\theta(x, u) - v_\psi^\theta(x) \\ &\approx q_\varphi^\theta(x, u) - \mathbb{E}_{u \sim u_\theta(\cdot|x)} [q_\varphi^\theta(x, u)] \end{aligned} \quad (8.18)$$

is called the advantage function. It measures how much better the particular control u is for a state x as compared to the average return of controls sampled from the controller at that state. The form (8.17) is the most commonly implemented form in research papers whenever they say “we use the policy gradient”.

❓ The advantage function is very useful while doing theoretical work on RL algorithms. But it is also extremely useful in practice. It imposes a constraint upon our estimate q_φ^θ and the estimate v_ψ^θ . If we are not solving (8.14) and (8.16) to completion, we may benefit by imposing this constraint on the advantage function. Can you think of a way?

325 8.5.2 Discussion

326 Policy gradients are, in general, a complicated suite of algorithms to
327 implement. You will see some of this complexity when you implement
328 the controller for something as simple as the simple pendulum. The key
329 challenges with implementing policy gradients come from the following.

- 330 1. Need lots of data, each parameter update requires fresh data from
331 the systems. Typical problems may need a million trajectories, most
332 robots would break before one gets this much data from them if one
333 implements these algorithms naively.
- 334 2. The log-likelihood ratio trick has a high variance due to $u_\theta(\cdot | x)$

being in the denominator of the expression, so we need to implement complex variance reduction techniques such as actor-critic methods.

3. Fitting the Q-function and the value function is not easy, each parameter update of the policy ideally requires you to solve the entire problems (8.14) and (8.16). In practice, we only perform a few steps of SGD to solve the two problems and reuse the solution of k^{th} iteration update as an initialization of the $k + 1^{\text{th}}$ update. This is known as “warm start” in the optimization literature and reduces the computational cost of fitting the Q/value-functions from scratch each time.
4. The Q/value-function fitted in iteration k may be poor estimates of the Q/value at iteration $k + 1$ for the new controller $u_{\theta^{(k+1)}}(\cdot | x)$. If the controller parameters change quickly, $\theta^{(k+1)}$ is very different from $\theta^{(k)}$, then so are $q^{\theta^{(k+1)}}$ and $v^{\theta^{(k+1)}}$. There is a very fine balance between training quickly and retaining the efficiency of warm start; and tuning this in practice is quite difficult. A large number of policy gradient algorithms like TRPO (<https://arxiv.org/abs/1502.05477>) and PPO (<https://arxiv.org/abs/1707.06347>) try to alleviate this with varying degrees of success.
5. The latter, PPO, is a good policy-gradient-based algorithm to try on a new problem. For instance, in a very impressive demonstration, it was used to build an RL agent to play StarCraft (<https://openai.com/blog/openai-five>). We will see better RL methods in the next chapter.

8.6 Pre-conditioning the gradient in reinforcement learning

Newton’s method for solving a minimizing problem $\arg\min \ell(w)$ corresponds to weight updates

$$w^{(t+1)} = w^{(t)} - \left(\nabla^2 \ell(w^{(t)}) \right)^{-1} \nabla \ell(w^{(t)}). \quad (8.19)$$

Newton’s method converges quadratically, i.e., $\ell(w^{(t+1)}) - \ell(w^*) \leq c \left(\ell(w^{(t)}) - \ell(w^*) \right)^2$, the sub-optimality shrinks very quickly. It is however very difficult to implement Newton’s method for large optimization problems, such as deep networks, because the Hessian $\nabla^2 \ell(w) \in \mathbb{R}^{p \times p}$ is a very large matrix. Inverting such large matrices at each step of descent is very expensive and furthermore, for many real problems the Hessian is extremely ill-conditioned $\lambda_{\max}/\lambda_{\min} \approx 10^8$; in fact for deep networks which have two times as many weights as the number of weights the Hessian always has zero eigenvalues. In general, we can use any other pre-conditioning matrix in place of the inverse Hessian so long as it is positive semi-definite

$$w^{(t+1)} = w^{(t)} - \eta G^{-1/2} \nabla \ell(w^{(t)}); \quad (8.20)$$

▲ Pre-conditioning is the second classical trick to accelerating optimization.

▲ One can implement Newton’s method for some small-scale neural networks using Hessian-vector products.

the square root is taken to emphasize the positive definiteness. We can approximate the Hessian using the so-called Gauss-Newton matrix:

$$\nabla^2 \ell(w) \approx \frac{1}{n} \sum_{i=1}^n \nabla \ell^i(w) \nabla \ell^i(w)^\top \quad (8.21)$$

if the gradients $\nabla \ell^i(w) \approx 0$. In other words, the Hessian, up to an approximation that holds towards the end of training, is equal to the average of the outer products of the individual sample gradients. This is still a very large matrix, so let's use only the diagonal and set the off-diagonal elements to zero:

$$\sqrt{G_{ii}} \equiv (\nabla^2 \ell(w))_{ii} = \frac{1}{n} \sum_{i=1}^n \left(\frac{d\ell^i(w)}{dw_i} \right)^2,$$

i.e., the diagonal elements of the Hessian under this approximation are the averages of the squares of the gradients of the individual samples. We can now write the pre-conditioned descent direction as

$$\left(G^{-1/2} \nabla \ell(w^{(t)}) \right)_i \approx \frac{\frac{d\ell(w^{(t)})}{dw_i}}{\sqrt{\frac{1}{n} \sum_{i=1}^n \left(\frac{d\ell^i(w)}{dw_i} \right)^2}}. \quad (8.22)$$

When implementing this descent direction using stochastic gradients, we can calculate the denominator using a run-time average of the square of the gradients

$$\frac{1}{t} \sum_{s=1}^t \nabla_i \ell^{\omega_s}(w^{(s)})$$

Adam implements a pre-conditioned version of the gradient descent equation using a (square root of the) diagonal approximation of the Hessian as the pre-conditioning matrix. The new descent direction is (8.22) where again the denominator is calculated using exponential averaging instead of the Euclidean average. This is very useful to train deep networks in general because the gradients tend to ill-conditioned. This is particularly useful in reinforcement learning because the controller and the value function objectives have very different magnitudes, the former is equal to the average return while the latter is often near zero because it is the Bellman error.

8.6.1 The Levenberg-Marquardt algorithm

As we said, in many real problems when neural networks are being fitted, the Hessian tends to be very ill-conditioned. In such situations, it is useful to damp the iterations by taking a kind of average between gradient descent and (8.20) which uses the Gauss-Newton matrix. Consider an

392 optimization problem given by

$$w^* = \underset{w}{\operatorname{argmin}} \ell(w) \doteq \sum_{i=1}^N (y^i - f(x^i, w))^2.$$

393 where $f(x^i, w)$ is the output on the i^{th} datum. In RL this would be, say,
 394 one of the elements in the average over the Bellman error. From an iterate
 395 $w^{(t)}$, if we are making an update Δw to the weights, we should have

$$f(x^i, w^{(t)} + \Delta w) = f(x^i, w^{(t)}) + J^i(w^{(t)})\Delta w$$

396 using the Taylor series where the Jacobian $\mathbb{R}^p \ni J^i(w^{(t)}) = \frac{\partial f(x^i, w)}{\partial w} \big|_{w=w^{(t)}}$.
 397 In other words,

$$\begin{aligned} \ell(w + \Delta w) &= \sum_{i=1}^N (y^i - f(x^i, w + \Delta w))^2 \\ &= \left\| \vec{y} - \vec{f}(w) - J\Delta w \right\|^2 \\ &= \left(\vec{y} - \vec{f}(w) \right)^\top \left(\vec{y} - \vec{f}(w) \right) - 2 \left(\vec{y} - \vec{f}(w) \right)^\top J\Delta w + \Delta w^\top J^\top J\Delta w. \end{aligned}$$

The Levenberg-Marquardt algorithm finds the best step Δw that leads to maximal reduction in the objective $\ell(w)$ using a locally quadratic approximation of the objective. By definition, such a step should always lead to improvement in the objective.

398 If we take the derivative of the right-hand side with respect to Δw to
 399 calculate the best step forward and minimize $\ell(w + \Delta w)$, we get

$$(J^\top J) \Delta w = J^\top (\vec{y} - \vec{f}(x)). \quad (8.23)$$

400 This is a linear equation for the step Δw that can be solved by inverting
 401 $J^\top J$, or using a damped version which looks like

$$(J^\top J + \lambda I) \Delta w = J^\top (\vec{y} - \vec{f}(x)).$$

402 The (non-negative) damping factor λ is adjusted at each iteration. If
 403 reduction of the loss is rapid, a smaller value can be used, bringing the
 404 algorithm closer to the Gauss–Newton algorithm, whereas if an iteration
 405 gives insufficient reduction in the residual, λ can be increased, giving a
 406 step closer to the gradient-descent direction.

This is called the Levenberg-Marquardt algorithm and the region

defined by the damping parameter λ is called the trust-region. Effectively we trust our local approximation using the Taylor series within a ball of size λ . This is one of the most powerful optimization algorithms one can use, much better than SGD or Adam because it uses information about the curvature. It is easy to use when the Jacobian can be calculated, e.g., for small problems or problems with small neural networks. You can also use it from within scipy.

8.6.2 Trust region policy optimization (TRPO)

We will next discuss a trust-region-based algorithm for policy gradient. Consider two controllers u_θ and $u_{\theta'}$. Kakade and Langford proved in a 2002 paper titled “Approximately Optimal Approximate Reinforcement Learning” a very beautiful result. This is the basis of the TRPO algorithm and a later one called PPO which improves its practical performance. This paper showed that

$$v^\theta(x) = v^{\theta'}(x) + \underbrace{\mathbb{E}_{\tau \sim p_\theta}}_{\text{new controller}} \underbrace{\left[\sum_t \gamma^t a^{\theta'}(x_t, u_t) \right]}_{\text{old controller's advantage}} \quad (8.24)$$

for all trajectories τ that begin at state x . In words, the average return of one controller u_θ can be calculated using trajectories collected from this new controller as the discounted averaged of the the advantage of the old controller. We can rewrite this using the discounted state visitation frequency as

$$v^\theta(x) = v^{\theta'}(x) + \mathbb{E}_{x \sim d^\theta, u \sim u_\theta} \left[a^{\theta'}(x, u) \right].$$

This is quite nice because given an old controller $u_{\theta'}$, if we can ensure that the new one u_θ takes controls where the advantage of the old controller was positive, at each state,

$$\mathbb{E}_{u \sim u_\theta} \left[a^{\theta'}(x, u) \right] \geq 0,$$

(or at least on average over the states), then the new controller has a better average reward.

We could use this property to improve the current controller. You can think of this as the property satisfied by each iteration of policy iteration algorithm, the new policy’s controller is better than the old policy’s controller in at least one of the states of the MDP.

If we cannot find this better controller u_θ then we have converged to the optimal policy.

426 In practice, we estimate the advantage and the state visitation frequency
 427 using samples. Therefore, it is not easy to implement the idea above. To
 428 work around this, Kakade and Langford suggested a clever idea that is
 429 a local approximation of this objective (conceptually very similar to the
 430 Taylor series approximation we took above in Levenberg-Marquardt)

$$v^\theta(x) = v^{\theta'}(x) + \underbrace{\mathbb{E}_{x \sim d^{\theta'}, u \sim u_\theta}}_{\text{old visitation frequency}} \left[a^{\theta'}(x, u) \right]. \quad (8.25)$$

431 Notice the subtle change where the states are now sampled from the
 432 visitation frequency of the old controller. Certainly if $u_\theta \approx u_{\theta'}$ for all
 433 states, then the two are similar.

434 Just like we use a trust-region in Levenberg-Marquardt to control the
 435 changes in the weights using a local approximation, we use a trust region
 436 in TRPO because we do not know much of a step to take when we update
 437 this old controller using this local approximation of the objective. We
 438 used a ball in the weight space of size λ to control the step. In RL, we
 439 can use some notion of the discrepancy between the old controller and the
 440 updated controller. In TRPO, this is done using some divergence between
 441 the controls sampled from the two controllers at different states, e.g.,

$$\mathbb{E}_{x \sim d^{\theta'}} [\text{KL}(u_\theta(\cdot | x), u_{\theta'}(\cdot | x))].$$

442 This gives us the TRPO objective. At each iteration with weights θ' , it
 443 solves:

$$\begin{aligned} & \max_{\theta} \mathbb{E}_{x \sim d^{\theta'}} \mathbb{E}_{u \sim u_\theta} \left[a^{\theta'}(x, u) \right] \\ & \text{subject to } \mathbb{E}_{x \sim d^{\theta'}} [\text{KL}(u_\theta(\cdot | x), u_{\theta'}(\cdot | x))] \leq \lambda. \end{aligned} \quad (8.26)$$

444 to obtain updated parameters.

How would we implement this objective? We would first sample trajectories using $u_{\theta'}$, to calculate the state-visitation frequency $d^{\theta'}$. Then at each state we should calculate the controls taken by the new controller u_θ . You should immediately see a problem with this... The new controls will send the system to different states than those in our trajectories from $u_{\theta'}$. This would not be a problem if we were calculating the state-visitation frequency exactly, but since we only use a few trajectories, we would not know how likely the new state was under the old controller.

445 To work around this issue, we can use importance sampling which we
 446 have seen before in particle filtering. We rewrite the expectation using the

i We can also implement TRPO in (8.26) using Levenberg-Marquardt to obtain the trust region instead of a constraint on the changes to the controller. We need to use a trust-region formed by a non-isotropic ball in the weight space of the controller instead of a simple λI because the output of the controller, namely the state-visitation frequency and the controller's probabilities have very different sensitivities to changes in different parameters of the neural network. Instead of worrying about what this non-isotropic ball is in the weight space, it is easy to control the step directly in the controller. The two are equivalent.

ratio of the probabilities of the two controllers:

$$\max_{\theta} \underbrace{\mathbb{E}_{x \sim p_{\theta'}}}_{\text{old controller}} \underbrace{\mathbb{E}_{u \sim u_{\theta'}}}_{\text{importance ratio}} \left[\frac{u_{\theta}(u | x)}{u_{\theta'}(u | x)} a^{\theta'}(x, u) \right]. \quad (8.27)$$

8.6.3 Proximal policy optimization (PPO)

TRPO was based on sound ideas and it also gives a proof that the value of the controller improves monotonically under certain conditions. The authors in the popular paper titled “Proximal Policy Optimization Algorithms” observed that this was not true in practice on some problems.

From our description of the algorithm, it should be clear when TRPO cannot make monotonic improvements upon the value. If the trust region parameter λ in (8.26) is too large, then we allow large changes to the controller. This can lead to very large differences in the state-visitation frequency $d^{\theta'}$ vs. d^{θ} . And all our mathematics of local approximations breaks down. If we use too small a parameter λ then the new controller is not allowed to be much different than the old one...and therefore the algorithm might appear slow in making progress. In Levenberg-Marquardt-type methods, the workaround to this issue is well-known. One should modify the parameter λ during the course of optimization adaptively—we cannot have it be fixed for the entire duration.

PPO implements two tricks to improve the practical performance of TRPO.

1. The first observation is that the importance ratio should be close to 1. If it is too large or too small, this indicates that the two controllers have very different probabilities for the controls taken by the old controller (and thereby the new controller’s state-visitation frequency is very different...). We therefore use a parameter ϵ to clip the likelihood-ratio to satisfy

$$1 - \epsilon \leq \frac{u_{\theta}(u | x)}{u_{\theta'}(u | x)} \leq 1 + \epsilon.$$

2. The trust region parameter λ is changed during the course of optimization. If the change in the controller after the weight update

$$\Delta = \mathbb{E}_{x \sim d^{\theta(t)}} [\text{KL}(u_{\theta(t+1)}(\cdot | x), u_{\theta(t)}(\cdot | x))]$$

is very small, say $\Delta < \Delta_{\text{user}}/2$, then we use 2λ for the next iteration, and if this change is too big, say $\Delta > 2\Delta_{\text{user}}$, then we decrease the trust region to set $\lambda/2$ for the next iteration. These parameters should be chosen carefully by watching the different parts of training objective. For example, scipy will implement a lot of careful choices to modify the Levenberg-Marquardt parameter and that is why it gives good answers for many nonlinear optimization algorithms.

The original PPO paper implements a few different things on top of this,

482 e.g., generalized advantage estimation, but many people have noticed that
483 these tricks are not important.

PPO is a very good algorithm for reinforcement learning and should be the “first” thing you try on a new problem. As we have seen in this chapter, it implements some very classical ideas. Unfortunately, when it comes to policy gradient methods, we need all the techniques discussed in this chapter to get meaningful answers, e.g., actor-critic methods, calculating advantage to do variance reduction, using Adam to update the weights of the neural networks, and using trust-region methods like PPO to select step-sizes. There is a lot of good code available to implement these things, e.g., Open AI’s Spinning Up <https://github.com/openai/spinningup> is a very good resource to study how to implement RL algorithms in < 500 lines of code. There are a lot of other fancier libraries but they do not work any better than this simple one.